

PROGRAMMING LANGUAGES FOUNDATIONS

Computer Science III

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Lecturer
Computer Engineering
School of Engineering
Universidad Distrital Francisco José de Caldas

2025-I



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

- 1 History of the Computation
- 2 Programming Languages
- 3 Finite-State Machines 
- 4 Generative Grammars



Outline

- 1 History of the Computation
- 2 Programming Languages
- 3 Finite-State Machines
- 4 Generative Grammars



Babbage Machine

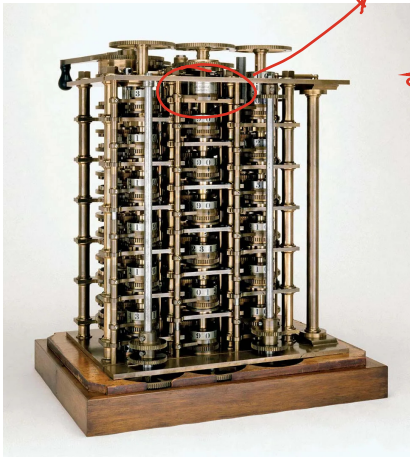


Figure: Analytical Machine

- **Charles Babbage** (1791 — 1871) was an English mathematician, philosopher, inventor, and mechanical engineer.
- He originated the **concept** of a digital programmable computer.
- Considered the “father of the computer”. He creates the Analytical Engine.
- The **Analytical Engine** was a general-purpose mechanical computer.



Ada Lovelace

- **Ada Lovelace** (1815 — 1852) was an English mathematician and writer.
- She is known for her work on Charles Babbage's early mechanical general-purpose computer, the Analytical Engine.
- She was the first to recognize that the machine had applications beyond pure calculation, and to have published the first algorithm intended to be carried out by such a machine.



Figure: Ada Lovelace



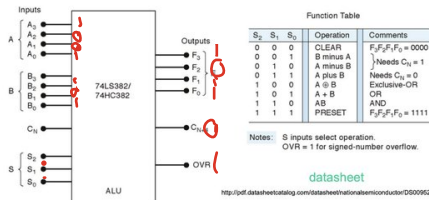
Physical Binary Language

- **Binary** is a **base-2** number system. It uses only **two** symbols: typically **0** (zero) and **1** (one).
- The **bit** is the **basic unit** of **information** in computing and digital communications.

→ CLW

Arithmetic Logic Unit

- Examine the functionality of this 74LS382 ALU chip



ECE331: MIPS Instructions-1 13

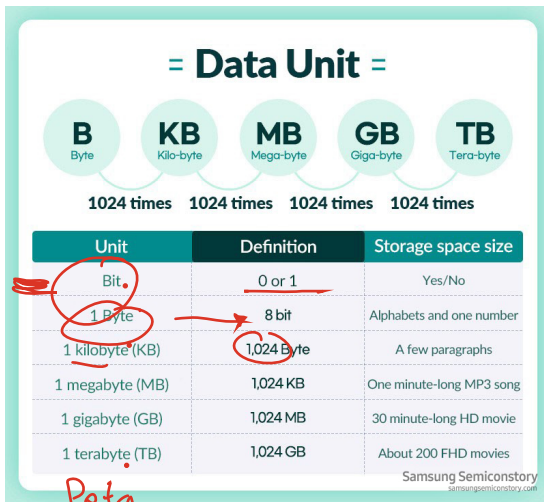


Bits and Bytes

$$2^{10} = 1024$$

$$2^8 = 256$$

ASCII
0-255



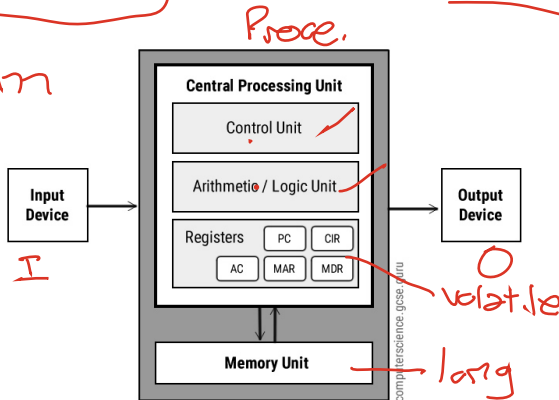
Peta



Von Neumann Architecture

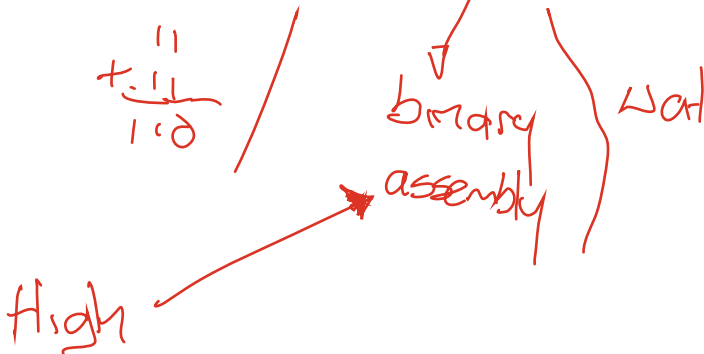
- The **Von Neumann Architecture** is a computer architecture based on the stored-program computer concept.
- The **design** is based on the concept of an instruction set.
- The program and data are both stored in the same memory unit.

System



Bit Operations

- **Bitwise operations** are operations that directly manipulate bits.
- They are used in low-level programming for performing calculations, file processing, and data compression.



Memory and Bit Storage



int

 $x = 4$

short, int 4 bytes
4 bytes

0000000000000000...0100
2m

$x \rightarrow 0x10$

→ HEAP-SIZE

Garbage Collector → scheduled



Outline

- 1 History of the Computation
- 2 Programming Languages**
- 3 Finite-State Machines
- 4 Generative Grammars



Assembly Programming Language

$\{ \text{if } (\text{eax} == 10) \{$

1 ; Example of a basic conditional structure in x86 assembly

2 $r1 \leftarrow 0x00$

3 cmp eax, 10 ; Compare the value in eax with 10

4 je equal_label ; Jump to equal_label if eax is equal to 10

5 .

6 ; Code for not equal case

7 jmp end_label ; Jump to end_label to avoid executing the equal case code

8

9 equal_label:

10 ; Code for equal case

11

12 end_label:

13 ; Continue execution

14

+ ultra speed

- ugly

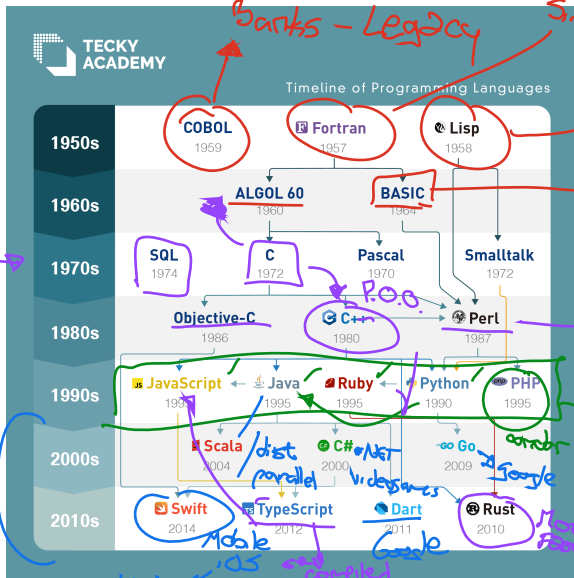
Processor unit

Machine & binary

Listing 1: Basic Conditional Structure in Assembly



History of Programming Languages



Me! & Fury

50 years ago



Ketlin



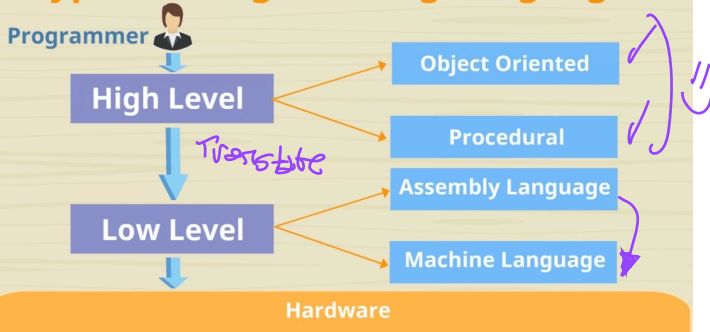
High-Level Programming Languages I

High-Level vs Low-Level

Nice Syntax

Performance

Types of Programming Languages



inprogrammer



High-Level Programming Languages II

Purpose of High-Level Languages

Ease of Use

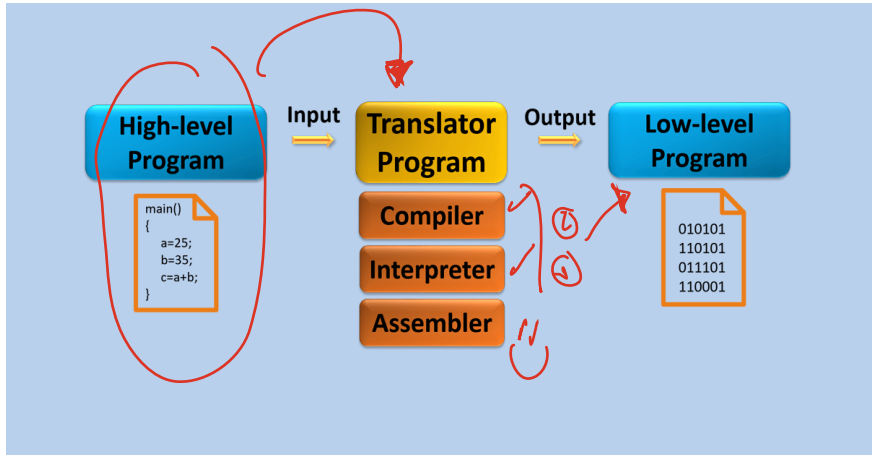
- Simplifying programming
- Minimizing learning curve
- Enhancing productivity
- Automated memory management
- Clear syntax
- Readability and maintainability



Portability Across Systems

- Cross-platform compatibility ✓
- Utilization of compilers and interpreters ✓
- Seamless execution on various platforms ✓
- Reduction of platform-specific modifications ✓
- Enhanced flexibility across environments ✓

Translation Process



Efficiency and Readability

- **Efficiency** is the ability to avoid wasting materials, energy, efforts, money, and time in doing something or in producing a desired result.

- **Readability** is the ease with which a human reader can understand the purpose, control flow, and operation of source code.

Time Memory

C++ } 0ms
C

Python / 60ms

Java / 120ms



Efficiency and Readability

- **Efficiency** is the ability to avoid **wasting materials**, **energy**, efforts, **money**, and **time** in doing something or in producing a desired result.
- **Readability** is the ease with which a **human reader** can **understand** the **purpose**, **control flow**, and operation of **source code**.

~ Gx29,

~ ~

~ ~

~

Assembly

Pqkn...

x=5



Programming Languages Levels

32-bit (4-byte) ADD instruction:

100000	00101	00010	00011	000000000000
opcode	rc	ra	rb	(unused)

Could be something like: $\text{Reg}[4] \leftarrow \text{Reg}[2] + \text{Reg}[3]$

In assembler:

1 **ADD**(R2, R3, R4)

Low

In any high-level language like C:

1 $a = b + c;$

High



Interpretation

- **Interpretation** is the process of **executing** a program in a high-level language by another program.

- Interpretation is an effective implementation strategy when performing computation and/or when exploring

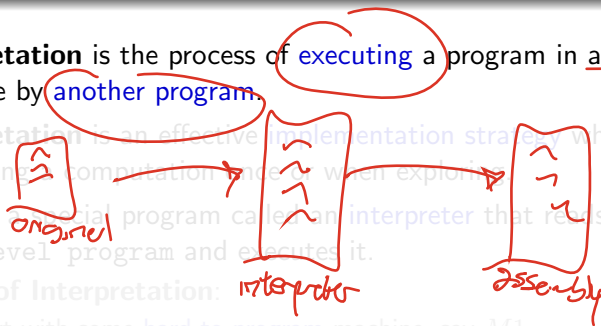
- There is a special program called an **interpreter** that reads a high-level program and executes it.

- **Model of Interpretation:**

- Start with some hard-to-program machine, say $M1$.
- Write a program $P1$ for $M1$ that mimics the operation of another easier machine $M2$.
- Result, $P1$ is an interpreter for $M2$, it means, a virtual $M2$.

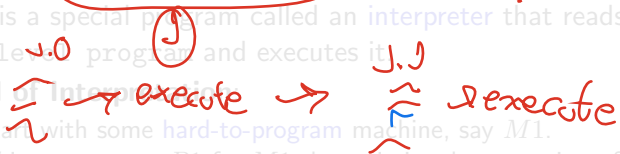
- **Advantages:**

- Portability.
- Flexibility.
- Ease of debugging.



Interpretation

- **Interpretation** is the process of **executing** a program in a high-level language by **another program**.
- **Interpretation** is an effective **implementation strategy** when performing a computation once or when exploring.
- There is a special program called an **interpreter** that reads a high-level program and executes it.
- **Model of Interpretation**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P1$ for $M1$ that mimics the operation of another easier machine $M2$.
 - Result, $P1$ is an **interpreter** for $M2$, it means, a **virtual** $M2$.
- **Advantages:**
 - Portability.
 - Flexibility.
 - Ease of debugging.



Interpretation

- **Interpretation** is the process of **executing** a program in a high-level language by **another program**.
- **Interpretation** is an effective **implementation strategy** when performing a computation once or when exploring.
- There is a special program called an **interpreter** that reads a **high-level program** and **executes it**.
- **Model of Interpretation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P1$ for $M1$ that mimics the operation of another easier machine $M2$.
 - Result, $P1$ is an **interpreter** for $M2$, it means, a **virtual** $M2$.
- **Advantages:**
 - Portability.
 - Flexibility.
 - Ease of debugging.



Interpretation

- **Interpretation** is the process of **executing** a program in a high-level language by **another program**.
- **Interpretation** is an effective **implementation strategy** when performing a computation once or when exploring.
- There is a special program called an **interpreter** that reads a high-level program and executes it.
- **Model of Interpretation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P1$ for $M1$ that **mimics** the operation of another easier machine $M2$.
 - Result $P1$ is an **interpreter** for $M2$, it means, a **virtual** $M2$.
- **Advantages:**
 - Portability.
 - Flexibility.
 - Ease of debugging.



Interpretation



- **Interpretation** is the process of **executing** a program in a high-level language by **another program**.
- **Interpretation** is an effective **implementation strategy** when performing a computation once or when exploring.
- There is a special program called an **interpreter** that reads a high-level program and executes it.
- **Model of Interpretation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P1$ for $M1$ that mimics the operation of another easier machine $M2$.
 - Result, $P1$ is an **interpreter** for $M2$, it means, a **virtual** $M2$.
- **Advantages:**
 - Portability. ✓
 - Flexibility. ✓
 - Ease of debugging. ✓



Compilation

- **Compilation** is the process of translating a program in a high-level language into a low-level language.
- **Compilation** is an effective **implementation strategy** when performing a computation many times.
- There is a special program called a **compiler** that reads a **high-level** program and translates it.
- **Model of Compilation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P2$ for $M1$ that translates a program in a high-level language into a program in a low-level language.
 - Result, $P2$ is a compiler for $M1$.
- **Advantages:**
 - Fast Execution.
 - Efficiency.
 - Portability.



Compilation

- **Compilation** is the process of **translating** a program in a high-level language into a **low-level language**.
- **Compilation** is an effective **implementation strategy** when performing a computation many times.

- There is a special program called a **compiler** that reads a high-level program and translates it.

- **Model of Compilation:**

- Start with some **hard-to-program** machine, say $M1$.

- Write a program $P2$ for $M1$ that translates a program in a high-level language into a program in a low-level language.

- Result, $P2$ is a **compiler** for $M1$.

- **Advantages:**

- Fast Execution.
- Efficiency.
- Portability.

PyCache / main.py

main.py



Compilation

- **Compilation** is the process of **translating** a program in a high-level language into a **low-level language**.
- **Compilation** is an effective **implementation strategy** when performing a computation many times.
- There is a special program called a **compiler** that reads a high-level program and translates it.
- **Model of Compilation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P2$ for $M1$ that translates a program in a high-level language into a program in a low-level language.
 - Result, $P2$ is a **compiler** for $M1$.
- **Advantages:**
 - **Fast Execution.**
 - **Efficiency.**
 - **Portability.**

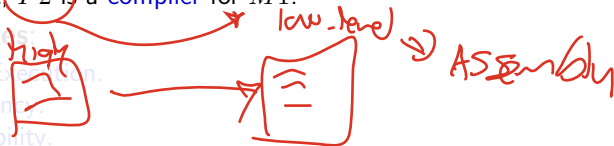


Compilation

- **Compilation** is the process of **translating** a program in a high-level language into a **low-level language**.
- **Compilation** is an effective **implementation strategy** when performing a computation many times.
- There is a special program called a **compiler** that reads a high-level program and translates it.
- **Model of Compilation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P2$ for $M1$ that **translates** a program in a high-level language into a program in a low-level language.
 - Result $P2$ is a **compiler** for $M1$.

Advantages:

- Fast Execution.
- Efficiency.
- Portability.



Compilation

- **Compilation** is the process of **translating** a program in a high-level language into a **low-level language**.
- **Compilation** is an effective **implementation strategy** when performing a computation many times.
- There is a special program called a **compiler** that reads a high-level program and translates it.
- **Model of Compilation:**
 - Start with some **hard-to-program** machine, say $M1$.
 - Write a program $P2$ for $M1$ that translates a program in a high-level language into a program in a low-level language.
 - Result, $P2$ is a **compiler** for $M1$.
- **Advantages:**
 - **Fast Execution**
 - **Efficiency**
 - **Portability**

Pointers



Interpretation Vs. Compilation

Characteristics differences:

	Compilation	Interpretation
How does it treat input $x + 2$?	Generate a program that computes $x + 2$	Computes $x + 2$
When it happens?	Before Execution	During Execution
What it complicates/slows?	Program Development	Program Execution
Decisions made at	Compile Time	Run Time

Assembly

Assembly
low
high

☺

C.V.
Spanish

official translation

① process



C.V.
English



Outline

- 1 History of the Computation
- 2 Programming Languages
- 3 Finite-State Machines**
- 4 Generative Grammars



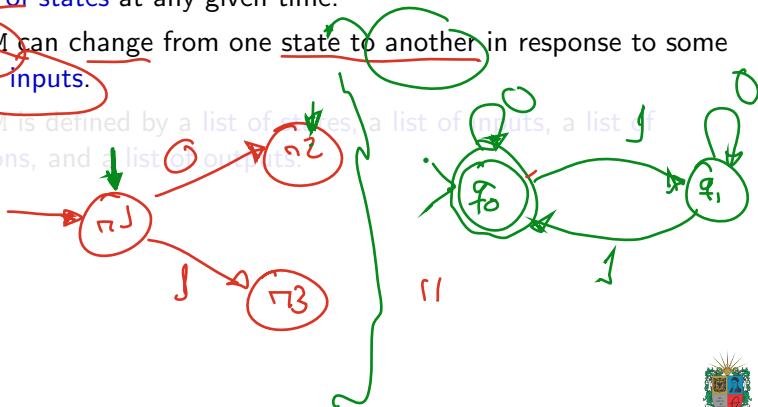
Finite-State Machines

- A **finite-state machine** (FSM) is a mathematical model of computation.
- It is an abstract machine that can be in exactly one of a finite number of states at any given time.
- The FSM can change from one state to another in response to some external inputs.
- The FSM is defined by a list of states, a list of inputs, a list of transitions, and a list of outputs.



Finite-State Machines

- A **finite-state machine** (FSM) is a **mathematical model** of computation.
- It is an **abstract machine** that can be in **exactly one of a finite number of states** at any given time.
- The FSM can change from one state to another in response to some external inputs.
- The FSM is defined by a list of states, a list of inputs, a list of transitions, and a list of outputs.



Finite-State Machines

- A **finite-state machine** (FSM) is a **mathematical model** of computation.
- It is an **abstract machine** that can be in **exactly one of a finite number of states** at any given time.
- The FSM can change from one state to another in response to some **external inputs**.
- The **FSM** is defined by a **list of states**, a **list of inputs**, a **list of transitions**, and a **list of outputs**.

states $\rightarrow q_0, q_1, q_2$

inputs $\rightarrow 0, 1$

$q_0, 0 \rightarrow q_0$

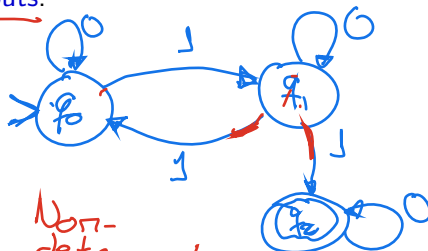
$q_0, 1 \rightarrow q_1$

$q_1, 0 \rightarrow q_1$

$q_1, 1 \rightarrow q_0$

$q_1, 1 \rightarrow q_2$

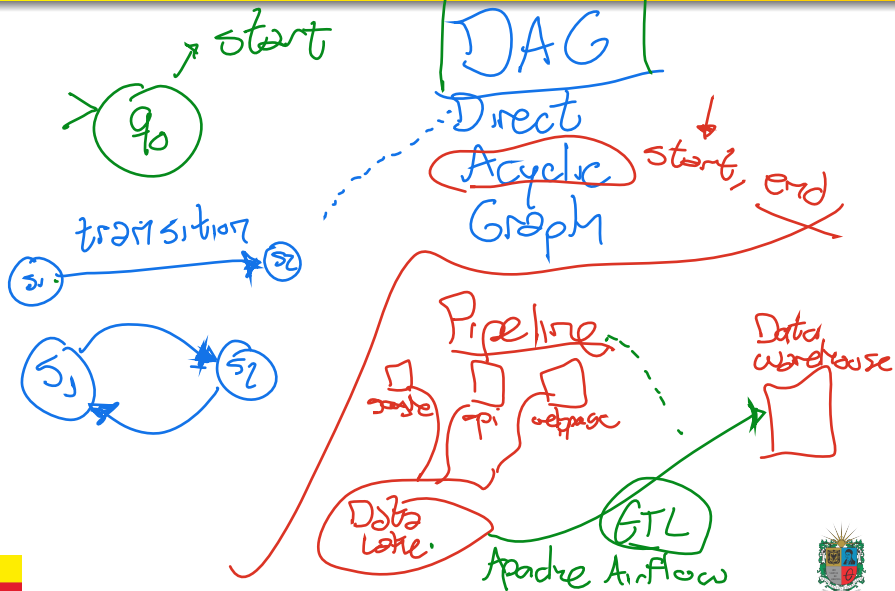
$q_2, 0 \rightarrow q_2$



Non-deterministic



Finite-State Machine Drawing



Alonzo Church

- **Alonzo Church** (1903 — 1995) was an American mathematician and logician.
- He is best known for the Lambda Calculus, which he developed in the 1930s.
- The Lambda Calculus is a formal system in mathematical logic for expressing computation based on function abstraction and application.

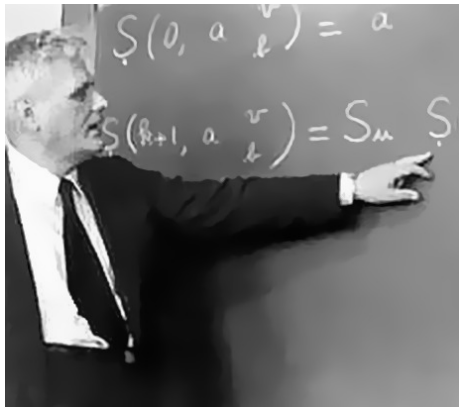


Figure: Alonzo Church



Regular Expressions

A regular expression is a sequence of characters that define a search pattern.

- **Regular expressions** are used in search engines, search and replace dialogs of word processors, and in text processing utilities.

- A regular expression is a pattern that is used to match character combinations in strings.
- The pattern describes one or more strings to match.

$[a-z]^{1,2,3} ([a-z][a-z]^*) [a-z]@distrital.edu.co$

Kleene's Star

X
J@ua--
✓ reperez@distrital.edu.co

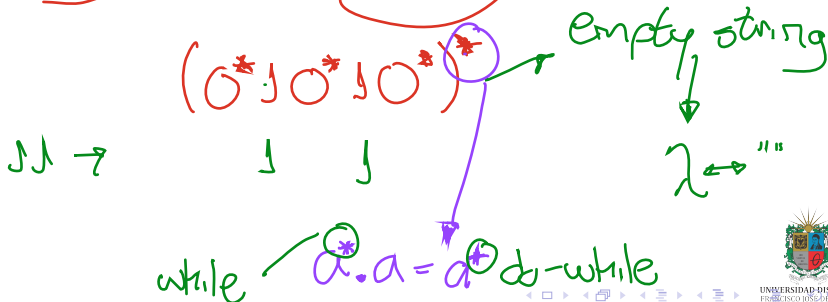
*.json



Regular Expressions

A **regular expression** is a **sequence of characters** that define a **search pattern**.

- **Regular expressions** are used in **search engines**, **search and replace dialogs** of **word processors**, and in **text processing utilities**.
- A **regular expression** is a **pattern** that is used to **match character combinations** in **strings**.
- The **pattern** describes one or more **strings** to match.



Finite Automata: Concatenation

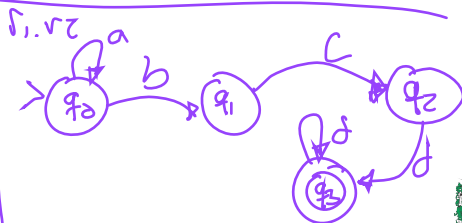
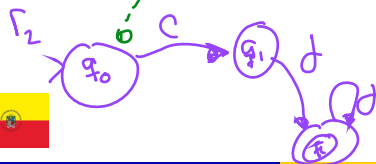
The **concatenation** of two regular expressions R_1 and R_2 is a regular expression that matches the **concatenation** of strings that are matched by R_1 followed by R_2 .

$$r_1 = a^*b$$

$$r_2 = cd^+$$

$$r_1 \cdot r_2 = a^*bcd^+$$

$$r_1 r_2 = a^*bcd^+$$



Finite Automata: Union

The **union** of two regular expressions R_1 and R_2 is a regular expression that matches the **union** of strings that are matched by R_1 **or** R_2 .

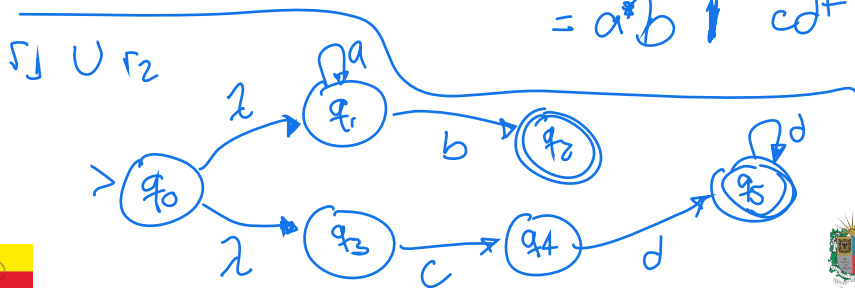
$$r_1 = a^*b$$

$$r_2 = cd^+$$

$$r_1 \cup r_2 = a^*b \cup cd^+$$

$$= a^*b \text{ or } cd^+$$

$$= a^*b \uparrow cd^+$$



Finite Automata: Kleene's Star

The **Kleene's star** of a **regular expression** R is a regular expression that matches the **concatenation** of **zero or more** strings that are matched by R .

$$\begin{aligned}
 a^* &= a \\
 a^2 &= a \cdot a \\
 a^0 &= \lambda
 \end{aligned}$$

$$a^* = a a a \dots a$$

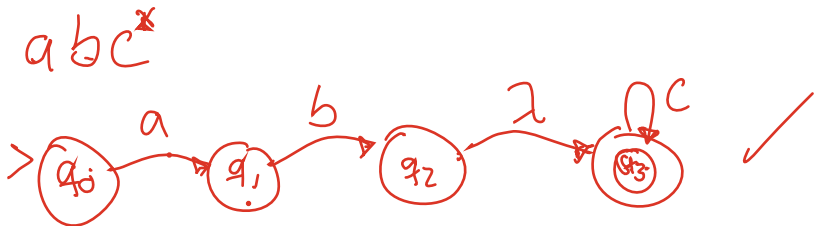
$$\begin{aligned}
 a^+ &= a \cdot a^* \\
 a^+ &= a^* \cdot a
 \end{aligned}$$

$$\begin{aligned}
 a \cdot \lambda &= a \\
 abc \cdot \lambda &= abc
 \end{aligned}$$

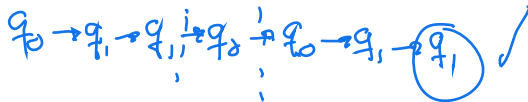
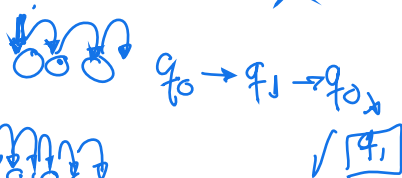
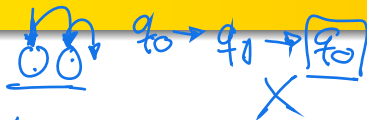
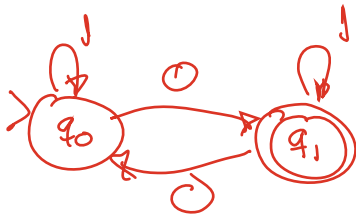


Finite Automata: λ -transition

The λ -transition is a transition that can be taken without consuming any input.



Strings Processing



Alan Turing

Church Turing

- Alan Turing (1912 — 1954) was an English mathematician, computer scientist, logician, cryptanalyst, philosopher, and theoretical biologist.
- He is widely considered to be the father of theoretical computer science and artificial intelligence.
- He was highly influential in the development of theoretical computer science, providing a formalization of the concepts of algorithm and computation with the Turing machine.

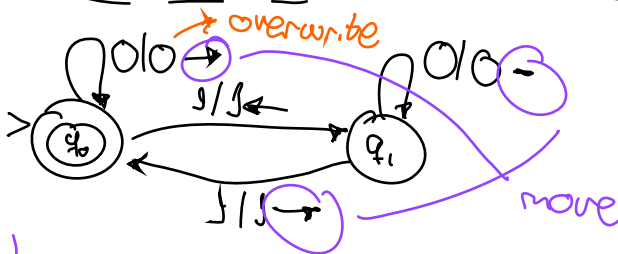


Figure: Alan Turing

parallel distributed binary



Turing Machine

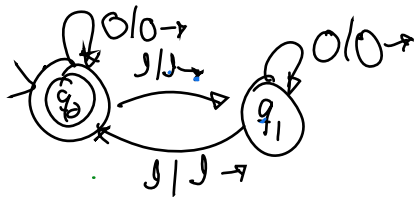


NP-Hard
NP-Complete



Universal Turing Machine

0101



$q_0 \rightarrow \cdot$ $q_1 \rightarrow \cdot$
 $0 \rightarrow \cdot$ $1 \rightarrow \cdot$
 $\rightarrow \cdot$ $\leftarrow \cdot$ $\rightarrow \cdot$

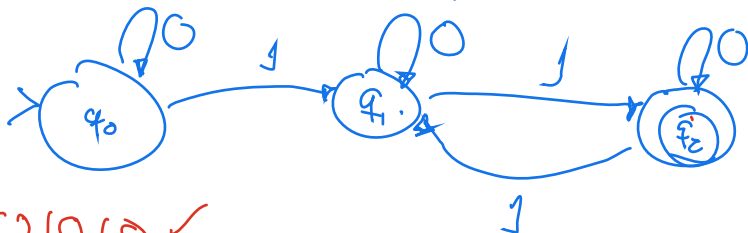
0010 10 1 0010 10 10 10 10 10 10 ...
 000 10 10 10 10 10 10



Regular Expression & Finite Automata: *Example i*

Be L a language over the alphabet $\Sigma = \{0, 1\}$, such that L is the set of all strings that contain an even number of 1s. The regular expression for L is:

$$L = (0^* 1 0^* 1 0^*)^*$$



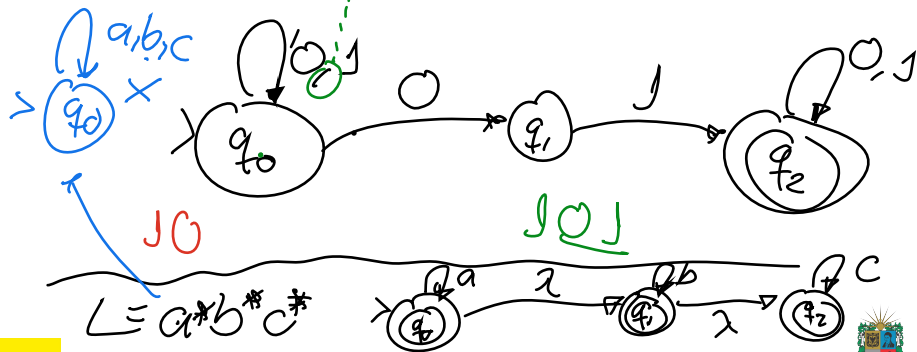
01010 ✓
 11 ✓
 11011 ✓



Regular Expression & Finite Automata: *Example ii*

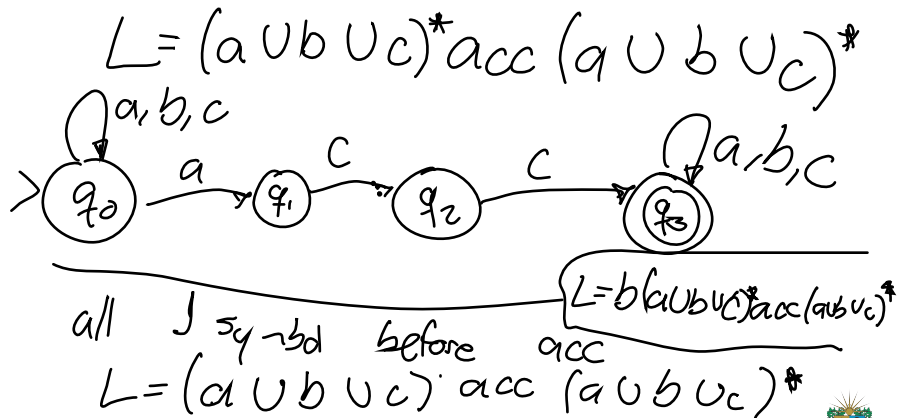
Be L a language over the alphabet $\Sigma = \{0, 1\}$, such that L is the set of all strings that contain the substring 01. The regular expression for L is:

$$L = (0 \cup 1)^* 0 1 (0 \cup 1)^*$$



Regular Expression & Finite Automata: *Example iii*

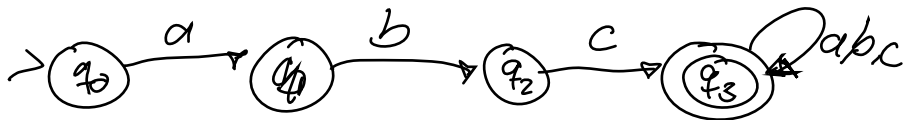
Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that L is the set of all strings that contain the substring acc . The regular expression for L is:



Regular Expression & Finite Automata: *Example iv*

Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that L is the set of all strings that start with the substring abc . The regular expression for L is:

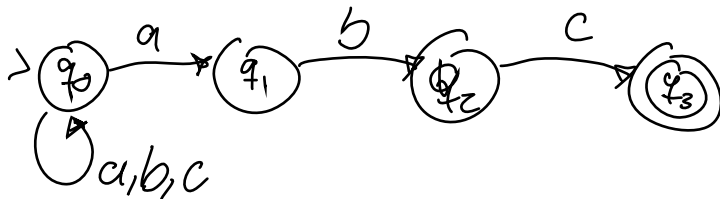
$$L = abc (a \cup b \cup c)^*$$



Regular Expression & Finite Automata: *Example v*

Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that L is the set of all strings that end with the substring abc . The regular expression for L is:

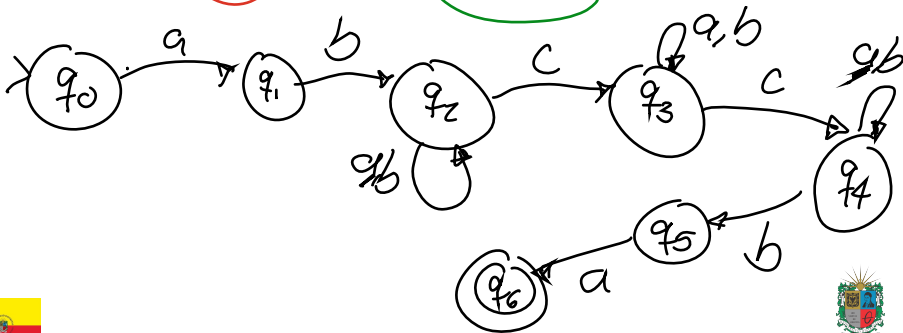
$$L = (a \cup b \cup c)^* abc$$



Regular Expression & Finite Automata: Example vi

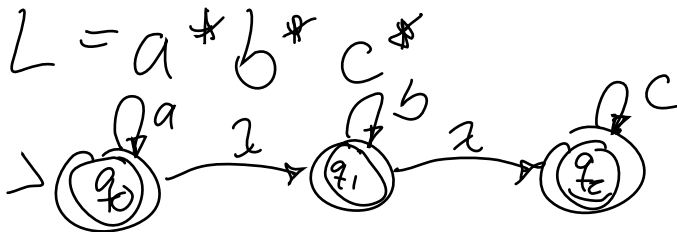
Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that L is the set of all strings that start with the substring ab , contain just two c 's and end with the substring ba . The regular expression for L is:

$$L = ab(a \cup b)^*c(a \cup b)^*c(a \cup b)^*ba$$



Regular Expression & Finite Automata: *Example vii*

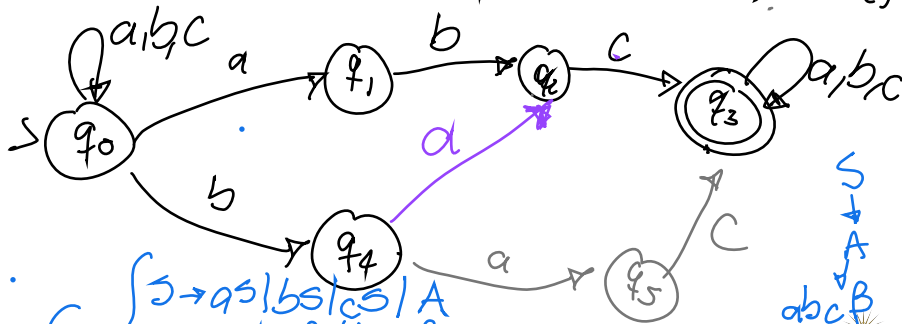
Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that L is the set of all strings that start with any number of a 's (could be 0), followed by any number of b 's, and end with any number of c 's. The regular expression for L is:



Regular Expression & Finite Automata: Example viii

Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that L is the set of all strings that contain the substring abc or bac . The regular expression for L is:

$$L = (a \cup b \cup c)^* (abc \cup bac) (a \cup b \cup c)^*$$



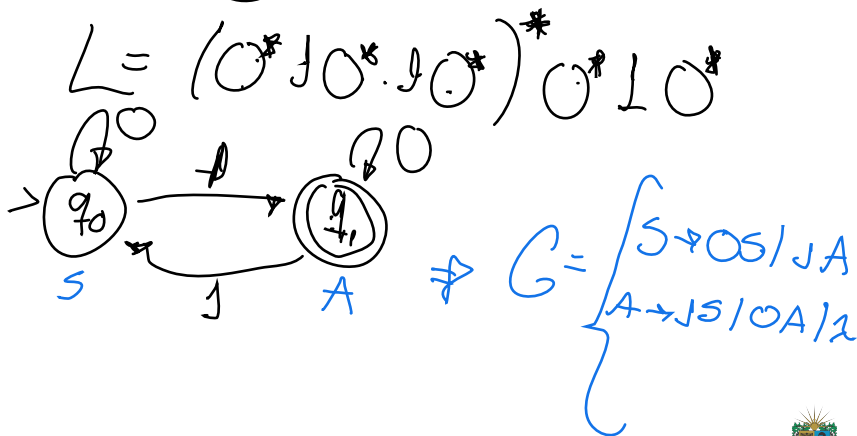
$$G = \begin{cases} S \rightarrow qS / bS / cS / \lambda \\ A \rightarrow abcB / bacB \\ B \rightarrow aB / bB / cB / \lambda \end{cases}$$

$S \rightarrow A$
 $A \rightarrow B$
 $B \rightarrow B$
 $B \rightarrow \lambda$



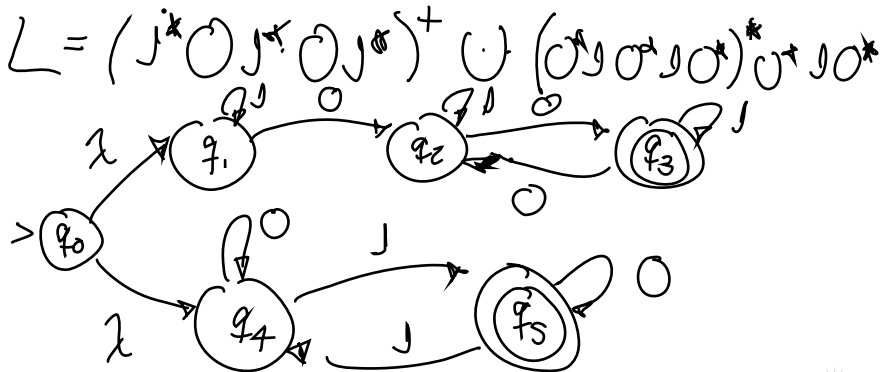
Regular Expression & Finite Automata: *Example ix*

Be L a language over the alphabet $\Sigma = \{0, 1\}$, such that L is the set of all strings that contain an odd number of 1s. The regular expression for L is:



Regular Expression & Finite Automata: *Example x*

Be L a language over the alphabet $\Sigma = \{0, 1\}$, such that L is the set of all strings that contain an even number of 0s or an odd number of 1s. The regular expression for L is:



Outline

- 1 History of the Computation
- 2 Programming Languages
- 3 Finite-State Machines
- 4 **Generative Grammars**



Noam Chomsky

- **Noam Chomsky** (1928 —) is an American linguist, philosopher, cognitive scientist, historian, social critic, and political activist.
- He is considered the father of modern linguistics.
- He introduced the **Chomsky hierarchy**, a classification of formal languages.

↓
structure



Figure: Noam Chomsky



Natural Processing Language

- **Natural Language Processing** (NLP) is a subfield of linguistics, computer science, information engineering, and artificial intelligence concerned with the interactions between computers and human languages.
 - NLP is used to apply algorithms to text and speech.
 - NLP is used to understand the meaning of text and speech.
 - NLP is used to generate human language text.
- vibe code*



Natural Processing Language

- **Natural Language Processing** (NLP) is a subfield of linguistics, computer science, information engineering, and artificial intelligence concerned with the interactions between computers and human languages.
- NLP is used to apply algorithms to text and speech.
- NLP is used to understand the meaning of text and speech.
- NLP is used to generate human language text.



Natural Processing Language

- **Natural Language Processing** (NLP) is a subfield of linguistics, computer science, information engineering, and artificial intelligence concerned with the interactions between computers and human languages.
- **NLP** is used to apply algorithms to text and speech.
- **NLP** is used to understand the meaning of text and speech.
- **NLP** is used to generate human language text.



Formal Languages

- A **formal language** is a set of **strings** of **symbols**. *words*
- The **alphabet** of a formal language is the set of **symbols** that can be used to form **strings**.
- A **formal grammar** is a set of **rules** for **generating strings** in a **formal language**.
- A **generative grammar** is a formal system for describing the structure of strings in a formal language.



Formal Languages

- A **formal language** is a set of **strings** of **symbols**.
- The **alphabet** of a formal language is the set of **symbols** that can be used to form **strings**.
- A **formal grammar** is a set of **rules** for **generating** strings in a **formal language**. *60's-70's*
- A **generative grammar** is a **formal system** for **describing** the **structure** of **strings** in a **formal language**.



Grammars Foundations

- A **grammar** is a set of **rules** for **generating strings** in a **formal language**.
- A **grammar** is a **formal system** for **describing the structure** of strings in a **formal language**.
- A **generative grammar** is a set of rules for generating strings in a formal language.

subject + V + C ant syst
c.nj.c.



Grammars Foundations

- A **grammar** is a set of rules for generating strings in a formal language.
- A **grammar** is a formal system for describing the structure of strings in a formal language.
- A **generative grammar** is a set of rules for generating strings in a formal language.



Chomsky Hierarchy

- The **Chomsky hierarchy** is a classification of formal languages.
- The **Chomsky hierarchy** is named after the linguist and cognitive scientist Noam Chomsky.
- The **Chomsky hierarchy** consists of four types of formal grammars:
 - Type 0: Unrestricted grammars.
 - Type 1: Context-sensitive grammars.
 - Type 2: Context-free grammars.
 - Type 3: Regular grammars.



Chomsky Hierarchy

- The **Chomsky hierarchy** is a **classification** of **formal languages**.
- The **Chomsky hierarchy** is named after the **linguist** and **cognitive scientist** **Noam Chomsky**.
- The **Chomsky hierarchy** consists of four types of **formal grammars**:
 - **Type 0**: **Unrestricted grammars**.
 - **Type 1**: **Context-sensitive grammars**.
 - **Type 2**: **Context-free grammars**.
 - **Type 3**: **Regular grammars**.

→ generalize



Context-Free Grammars

- A **context-free grammar** is a **formal grammar** and **generative grammar** in which every **production rule** is of the form:

$$A \rightarrow \alpha \cdot \quad (1)$$

replacement

- Where A is a nonterminal symbol and α is a string of terminals and nonterminals.
- A **context-free grammar** is a generative grammar that can generate a context-free language.

variable *replacement*

pronoun → I, you, he, she, it, they, we
verb → sleep, eat, run, jump, ...



Context-Free Grammars

- A **context-free grammar** is a **formal grammar** and **generative grammar** in which every **production rule** is of the form:

$$A \rightarrow \alpha \quad (1)$$

- Where A is a nonterminal symbol and α is a string of terminals and nonterminals.
- A **context-free grammar** is a **generative grammar** that can generate a **context-free language**.



Context-Free Grammars

- A **context-free grammar** is a **formal grammar** and **generative grammar** in which every **production rule** is of the form:

$$A \rightarrow \alpha \quad (1)$$

- Where A is a **nonterminal symbol** and α is a **string** of **terminals** and **nonterminals**.
- A **context-free grammar** is a **generative grammar** that can generate a **context-free language**.

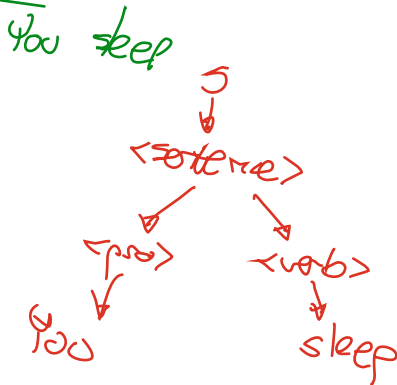
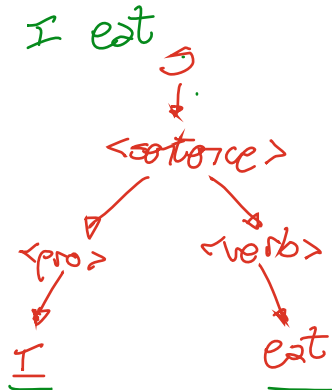
start

$$G = \begin{cases} S \rightarrow \langle \text{sentence} \rangle \\ \langle \text{sentence} \rangle \rightarrow \langle \text{pro} \rangle \langle \text{verb} \rangle \\ \langle \text{pro} \rangle \rightarrow I | you | they \\ \langle \text{verb} \rangle \rightarrow eat | sleep \end{cases}$$



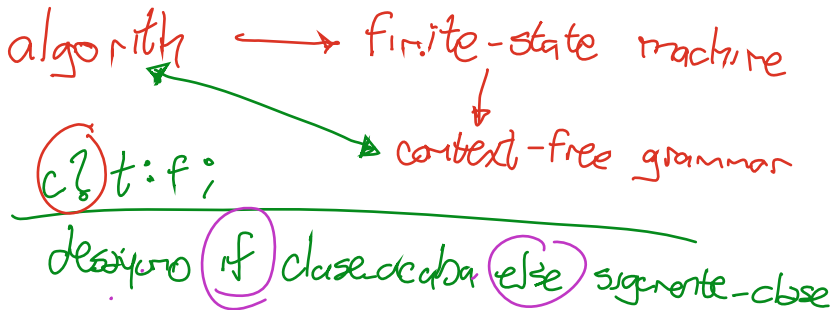
Derivation Trees

A **derivation tree** is a **tree** that represents the **sequence** of **production rules** used to **generate** a **string** in a **formal language**.



Equivalence between Grammars and Finite Automatas

A **context-free grammar** can be **converted** into a **finite automaton** and a **finite automaton** can be **converted** into a **context-free grammar**.



Equivalence Between Grammars and Regular Expressions

A regular expression can be converted into a finite automaton and a finite automaton can be converted into a regular expression.

regex $\rightarrow$ machine $\rightarrow$ grammar

$$L = a^i b^j a^i, \Sigma = \{a, b\}, i, j \geq 0$$

$$G = \begin{cases} S \rightarrow aSa / B \\ B \rightarrow bB / \lambda \end{cases}$$

$a^i b a^i$

$$a a b \lambda a a = a a b a a \quad a a b^2 a a$$



$a \underline{b} a$

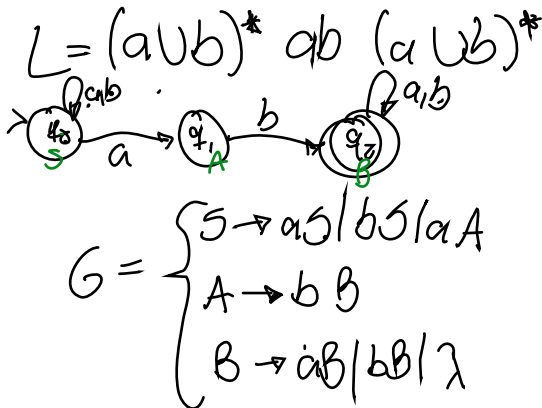
$a a \underline{b} a a$

$a a b^2 a a$



Free-Context Grammar: *Example i*

Be L a language over the alphabet $\Sigma = \{a, b\}$, such that $\underline{L}$ is the set of all strings that contain the substring $\underline{ab}$. The context-free grammar for L is:

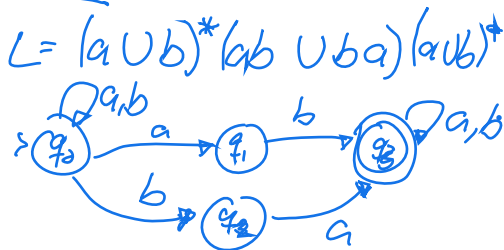


Free-Context Grammar: *Example ii*

Be L a language over the alphabet $\Sigma = \{a, b\}$, such that L is the set of all strings that contain the substring ab or ba . The context-free grammar for L is:

$$G = \begin{cases} S \rightarrow aS / bS / A \\ A \rightarrow abB / baB \\ B \rightarrow aB / bB / \lambda \end{cases}$$

$S \xrightarrow{a} aA$
 $\xrightarrow{b} bc$
 $\lambda \Rightarrow ab\lambda = ab$
 eg, valid



$$G = \begin{cases} S \rightarrow aS / bS / aA / bB \\ A \rightarrow bC \\ B \rightarrow aC \\ C \rightarrow aC / bC / \lambda \end{cases}$$

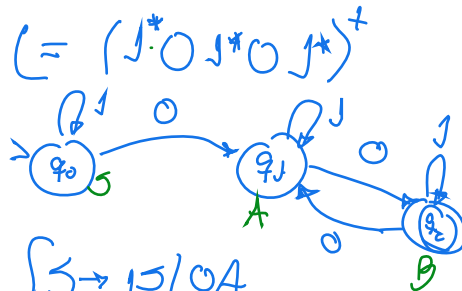


Free-Context Grammar: *Example iii*

11010010

Be L a language over the alphabet $\Sigma = \{0, 1\}$, such that L is the set of all strings that contain an even number of 0s. The context-free grammar for L is:

$$G = \begin{cases} S \rightarrow A0A0A / 2 \\ A \rightarrow 1A / 2 \end{cases}$$



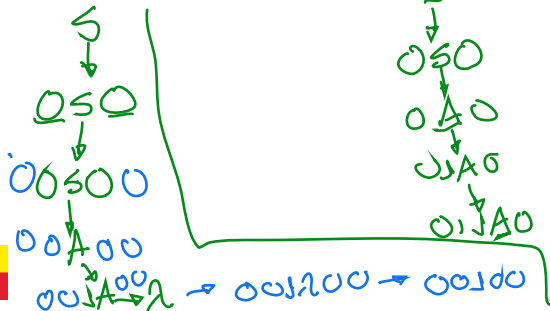
$$G = \begin{cases} S \rightarrow 1S / 0A \\ A \rightarrow 1A / 0B \\ B \rightarrow 1B / 0A / 2 \end{cases}$$



Free-Context Grammar: *Example iv*

Be L a language over the alphabet $\Sigma = \{0, 1\}$, such that $L = 0^i 1^j 0^i$, where $i, j \geq 0$. The context-free grammar for L is:

$$G = \begin{cases} S \rightarrow 0S0 / \lambda \\ A \rightarrow 1A / \lambda \end{cases}$$



00000
0110
00011...1000

0111A0
011120
01110



Free-Context Grammar: *Example v*

Be L a language over the alphabet $\Sigma = \{a, b, c\}$, such that $L = a^i b^j c^{i+j}$ where $i, j \geq 1$. The context-free grammar for L is:

$$G \rightarrow \begin{cases} S \rightarrow aSc / aAc \\ A \rightarrow bAc / bc \end{cases}$$

$abbccc$

$\frac{abc}{abc} \times$
 $\frac{abc}{abc} \times$



$abbccc$



Outline

$$G = \begin{cases} S \rightarrow \langle \text{sujeito} \rangle \langle \text{verbo} \rangle \langle \text{complemento} \rangle \\ \langle \text{sujeito} \rangle \rightarrow \text{Yo} / \text{Tu} / \text{Él} / \text{Ella} / \text{Nosotros} / \text{Ellos} \\ \langle \text{verbo} \rangle \rightarrow \text{comer} / \text{dormir} / \text{estudiar} \\ \langle \text{complemento} \rangle \rightarrow \text{mucho} / \text{poco} \end{cases}$$

3 Yo dormir poco

Él comer mucho

4 Nosotros estudiar poco



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-science-iii>

